

Introduction to Software Engineering

Dr Lily Sun

Learning Objectives

- ❑ Establish understanding of key aspects of software engineering
- ❑ Fundamentals of software engineering
- ❑ Systematic approach for engineering pipelines for a software development lifecycle
- ❑ Assurance for software quality towards correct, efficient, maintainable, scalable
- ❑ Collaboration among stakeholders, designers, developers, testers, maintainers
- ❑ Software lifecycle management – software evolution driven by business changes, trends triggered by new technologies (e.g., AI capabilities)

What is Software Engineering?

❑ A branch of computer science discipline

- ✿ Engineering discipline for building and maintaining software systems
- ✿ Focusing on
 - **reliability** – minimize bugs, crashes, and system failures
 - **scalability** – ensure stable performance as the number of users or system functionalities increases
 - **reusability** – develop modular, generalized code that can be reused across multiple projects
 - **extensibility** – allow for the easy addition of new features or modules without requiring major code changes
 - **user needs and expectations** – address user requirements by focusing on usability, accessibility, security, and data protection
 - **team collaboration** – effective communication among cross-functional teams (developers, testers, designers, stakeholders), and ensure robust version control (e.g., Git) and thorough documentation

What is Software Engineering? (cont.)

□ A branch of computer science discipline



.....



SE activities go beyond coding

- **planning** - requirements gathering, feasibility analysis, project estimation
- **design** – system architecture (functional components and their interface), UI (look-and-feel), UX (intuitive, user-friendly, and satisfying interactions), data modelling
- **testing** – unit testing, integration testing, system testing, bug tracking and fixing, user acceptance testing
- **deployment** - packaging and release, monitoring in production, rollback and recovery strategies, backup procedures
- **maintenance** - fixing bugs after release, updating for new requirements, enhancing performance or security
- **collaboration** - working in teams, version control and documentation, agile meetings and communication

Why Software Engineering Matters?

❑ Business perspectives

- 🌿 Solving real-world problems
- 🌿 Increasing efficiency and productivities
- 🌿 Driving innovation and evolution
- 🌿 Saving time and costs
- 🌿 Business push

❑ Software perspective

- 🌿 Building reliable and maintainable software
- 🌿 Constructing scalable software
- 🌿 Prevailing security, quality, legal procedures
- 🌿 Sustaining value-adding to business through evolution
- 🌿 Software application pull

Software Engineering vs Computer Science

Aspect	Computer Science (CS)	Software Engineering (SE)
Focus	Theoretical foundations of computing and algorithms	Practical application of engineering to software systems
Key Areas	Algorithms, data structures, theory of computation, AI principles	Software architecture, software methodology, design, implementation, testing, maintenance
Goal	Understand how computers work, internal resources optimisation	Build reliable, scalable, reusable and maintainable software
Approach	Exploratory, scientific, and experimental	Systematic, structured, and process-driven
Outputs	Scientific methods/models, prototypes, frameworks	Full software products and system applications
Core Skills	Problem solving, mathematical reasoning, logic, programming	Software lifecycle, application of computing knowledge and skills, project management, team collaboration
Example Topics	Theory of Computation, Machine Learning, Cryptography, Cloud Computing, Computer Networks	Agile Development, DevOps methodology, software modelling and testing
Career Roles	Systems Analyst/developer, Data Scientist, AI Researcher	Software Engineer (Prompt Engineer), System Architect, QA Engineer, DevOps Engineer, IT Consultant, Ethics Reviewer

Challenges in Software Engineering

❑ Changing User Requirements

- 🦋 user needs often evolve during the development process due to shifting business goals, market trends, or user feedback
- 🦋 late-stage changes can lead to scope creep, increased costs, the product not meeting actual user needs upon release
- 🦋 Waterfall methodology is linear and rigid, making leading to difficulties to accommodate changes without significant delays or rework

Challenges in Software Engineering (cont.)

□ Technical Responsibility

- 🦋 for meeting deadlines, teams might take shortcuts, e.g. writing quick, inefficient, or poorly documented code
- 🦋 over time, these compromises accumulate as technical debt, making the code bank harder to maintain, scale, or modify
- 🦋 increases the long-term cost of software maintenance and reduces development speed in the future

Challenges in Software Engineering (cont.)

□ Testing and Debugging Complexity

- 🦋 ensuring comprehensive testing becomes more difficult, while systems grow in size and complexity
- 🦋 manual testing is time-consuming and error-prone, while automated testing requires significant initial setup and maintenance
- 🦋 bugs may go undetected until after deployment, affecting software reliability and user trust

Challenges in Software Engineering (cont.)

❑ Scalability and Performance Issues

- 🦋 performance can degrade as more users or data are added, leading to slow response times or system crashes
- 🦋 poor scalability affects user experience and limits the system's ability to grow with demand

Challenges in Software Engineering (cont.)

❑ Time and Budget Constraints

- 🦋 projects often operate under strict deadlines and limited budgets
- 🦋 developers may be forced to deprioritise features, quality assurance, or documentation under time pressure
- 🦋 incomplete features, reduced testing, or a lack of polish in the final product, impacting reliability and user satisfaction

Introduction to Artificial Intelligence (AI) in SE

❑ AI capabilities

- 🌿 mimic human intelligence (learning, reasoning, adapting)

❑ Types of AI

- 🌿 Narrow AI (e.g., code suggestions)
- 🌿 Generative AI (e.g., ChatGPT, Copilot, etc.)
- 🌿 Autonomous systems (e.g., self-driving software)

AI Impacting Software Engineering

❑ Requirement Gathering

- 🌿 Use Natural Language Processing (NLP) to interpret user intent from requirement documents and user inputs

❑ Design

- 🌿 Leverage AI-assisted tools to suggest system architecture and functional features based on best practices and historical data

❑ Coding

- 🌿 Utilize AI-powered coding assistants like GitHub Copilot or Amazon CodeWhisperer to generate code based on algorithmic logic and developer context

AI Impacting Software Engineering (cont.)

☐ Testing

- 🦾 apply AI for automated test case generation, bug prediction, intelligent test coverage analysis to improve software quality
- 🦾 generate testing report and recommendations

☐ Maintenance

- 🦾 employ AI techniques for log analysis and anomaly detection
- 🦾 identify issues proactively and reduce downtime
- 🦾 generate maintenance task documents

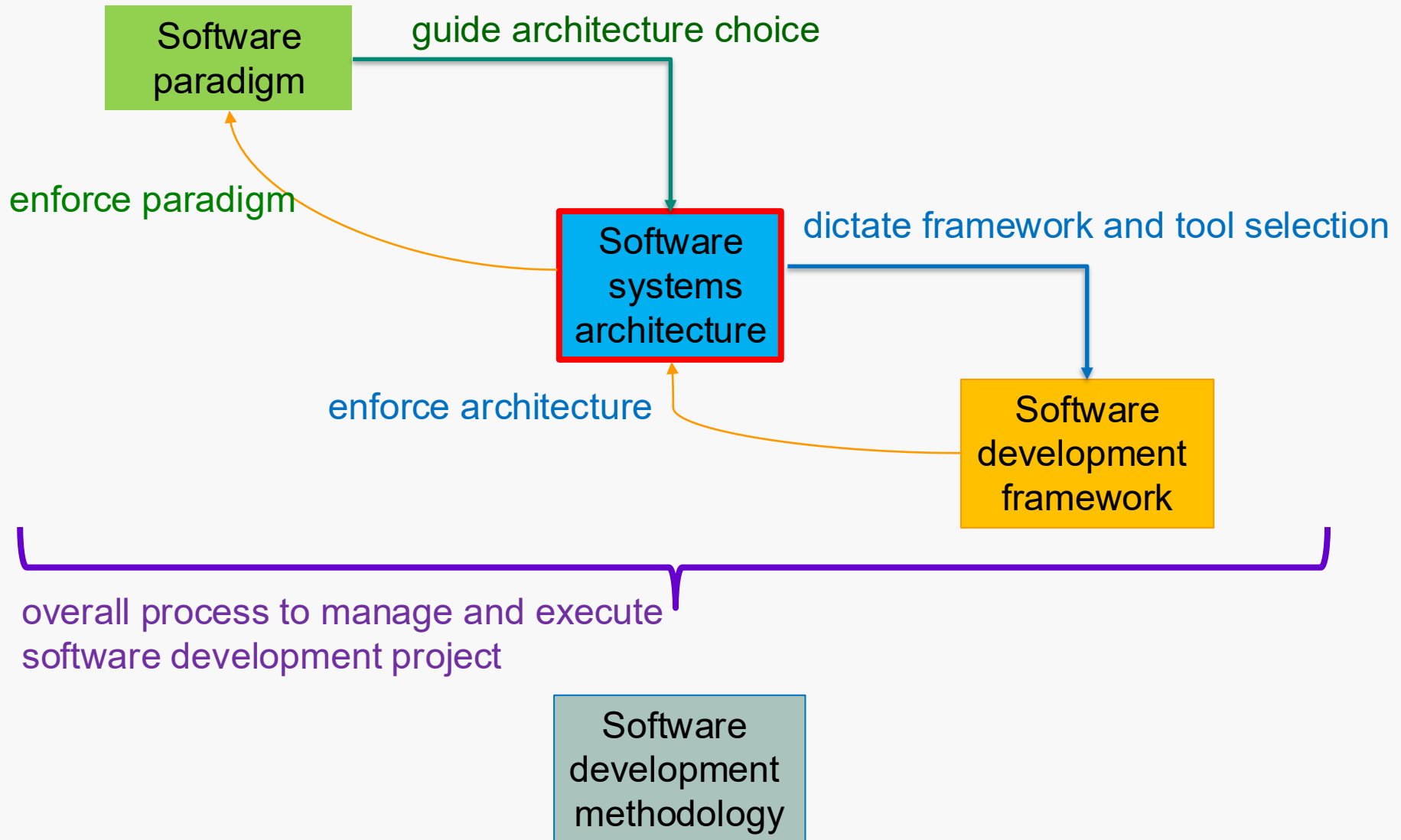
Key Messages to Take Away

- ❑ Software Engineering is a structured discipline for building robust software
- ❑ Software Engineering is more than just coding
 - 🌿 involving a structured process: planning, designing, building, testing, deploying, and maintaining software systems
 - 🌿 empowering businesses through AI + LLMs
- ❑ Teamwork and communication are essential
 - 🌿 successful software projects require collaboration across roles of stakeholders, designers, developers, testers, and managers

Benefits of AI in Software Engineering

- ❑ Speeds up development
- ❑ Improves code quality
- ❑ Reduces human error
- ❑ Enhances productivity
- ❑ Supports decision-making and debugging

SE Conceptual Flow



Conceptual Views

SE views	Depends On	Provides For
Paradigm	Conceptual philosophy	Guides design and thinking
Architecture	4+1 model: logical view, development view, process view, physical view and scenarios	Structures software system
Framework	Implements paradigm + architecture	Enables practical coding and reuse
Methodology	Waterfall, Agile, DevOps	Guides software development project execution and delivery

Conceptual Views Empowered by AI

SE views	Choice	Role
Paradigm	Modelling, testing, refactoring, static analysis	Approach/methodology for problem-solving (data process pipeline, AI) and programming (OOP)
Architecture	ArchiMate, TOGAF, UML tools,	Modular services: business service, application service, and technology service
Framework	React (frontend), Flask (backend), TensorFlow (AI)	Implements architecture, AI logic
Methodology	Agile + DevOps	CI/CD pipelines, gathering requirements, continuous feedback loops

Key Messages to Take Away (cont.)

❑ Next steps

- 🌿 Study the topics outlined throughout the semester
- 🌿 Conduct research by following the recommended materials and discussion threads
- 🌿 Engage in hands-on practical and tutorial exercises to deepen your theoretical understanding of key Software Engineering concepts, principles, methods, and techniques related to problem-solving
- 🌿 Continuously develop your Software Engineering knowledge and practical skills through the learning process, leading up to the assessments